First Hands-On LangChain Agent Exercise

Build a very small BMW diagnostic application using:

```
Streamlit
   ↓
FastAPI
   ↓
LangChain Agent
   ↓
OpenAI LLM
   ↓
get_vehicle_status()
   ↓
LLM
   ↓
Diagnosis
```

The purpose of this first lab is to understand only four LangChain concepts:

```
1. LLM
2. Tool
3. Agent
4. agent.invoke()
```

We will deliberately avoid databases, S3, RAG, LangGraph, memory, multiple agents, and complicated configuration.

Step 1 — Lab objective

Create an AI agent where the user asks:

```
Check BMW1001
```

The LLM should realize that it does not have vehicle telemetry itself and call:

Python Run

```python
get_vehicle_status("BMW1001")
```

The tool returns:

```
Battery: 12%
Temperature: 92°C
Fault Code: P0A80
```

The LLM then produces a diagnosis such as:

```
CRITICAL

Battery level is very low and the temperature
is extremely high.

Recommendation:
Stop driving safely and contact BMW service.
```

The important point is:

```
User does NOT directly call the tool.

LLM decides whether the tool is required.
```

Step 2 — Software required

You need:

```
Python 3.13
VS Code
OpenAI API key
Internet connection
```

Python packages:

```
langchain
langchain-openai
fastapi
uvicorn
streamlit
requests
python-dotenv
```

Step 3 — Create project

Create a folder:

```
bmw-langchain-lab
```

Open it in VS Code.

Your final project will be:

```
bmw-langchain-lab/
│
├── .env
├── pyproject.toml
│
├── agent.py
├── api.py
└── app.py
```

Only three Python files.

Step 4 — Create virtual environment

Open the VS Code terminal.

Windows:

```Bash
python -m venv .venv
```

Activate:

```Bash
.venv\Scripts\activate
```

You should now see something similar to:

```
(.venv) C:\bmw-langchain-lab>
```

Step 5 — Create `pyproject.toml`

Create:

```
pyproject.toml
```

Add:

```TOML
```

```
[project]

name = "bmw-langchain-lab"

version = "0.1.0"

requires-python = ">=3.13"

dependencies = [
    "langchain",
    "langchain-openai",
    "fastapi",
    "uvicorn",
    "streamlit",
    "requests",
    "python-dotenv"
]
```

Install:

Bash

```
pip install -e .
```

Verify:

Bash

```
pip list
```

You should find packages such as:

```
langchain
langchain-openai
fastapi
streamlit
uvicorn
```

Step 6 — Create `.env`

Create:

```
.env
```

Add:

env

```
OPENAI_API_KEY=your_openai_api_key
```

Do not put the API key directly inside Python.

The project now looks like:

```
bmw-langchain-lab/
│
├── .env
├── pyproject.toml
```

Step 7 — First create the tool

Create:

```
agent.py
```

Start with only this code:

Python Run

```python
from langchain.tools import tool


@tool
def get_vehicle_status(vehicle_id: str) -> str:
    """Get current BMW vehicle status."""

    if vehicle_id.upper() == "BMW1001":

        return """
        Vehicle: BMW1001
        Model: BMW iX
        Battery: 12%
        Temperature: 92 C
        Fault Code: P0A80
        """

    return "Vehicle not found"
```

The critical part is:

Python Run

```
@tool
```

This converts a normal Python function into a tool that can be made available to the agent.

Without LangChain you might have:

Python Run

```
def get_vehicle_status(vehicle_id):
    ...
```

With LangChain:

Python Run

```
@tool
def get_vehicle_status(vehicle_id: str):
    ...
```

Conceptually:

```
Normal Python Function
        ↓
      @tool
        ↓
LangChain Tool
        ↓
Available to the LLM Agent
```

Step 8 — Test the tool separately

Before creating the agent, verify the business function.

Temporarily add:

Python Run

```
print(
    get_vehicle_status.invoke(
        {"vehicle_id": "BMW1001"}
    )
)
```

Run:

Bash

```
python agent.py
```

Expected output:

```
Vehicle: BMW1001
Model: BMW iX
Battery: 12%
Temperature: 92 C
Fault Code: P0A80
```

Then remove the temporary `print()`.

This proves:

```
Tool works independently
```

before involving an LLM.

Step 9 — Add the LLM

Now modify `agent.py`.

Add:

Python Run

```python
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
```

Load `.env`:

Python Run

```python
load_dotenv()
```

Create the LLM:

Python Run

```python
llm = ChatOpenAI(
    model="gpt-5.6"
)
```

At this point:

```
ChatOpenAI
    ↓
OpenAI LLM
```

You have an LLM, but you do **not yet have an agent**.

Your code so far:

Python Run

```python
from dotenv import load_dotenv

from langchain.tools import tool
from langchain_openai import ChatOpenAI


load_dotenv()


@tool
def get_vehicle_status(vehicle_id: str) -> str:
    """Get current BMW vehicle status."""

    if vehicle_id.upper() == "BMW1001":

        return """
        Vehicle: BMW1001
        Model: BMW iX
        Battery: 12%
        Temperature: 92 C
        Fault Code: P0A80
        """

    return "Vehicle not found"


llm = ChatOpenAI(
    model="gpt-5.6"
)
```

Step 10 — Create LangChain Agent

Import:

Python Run

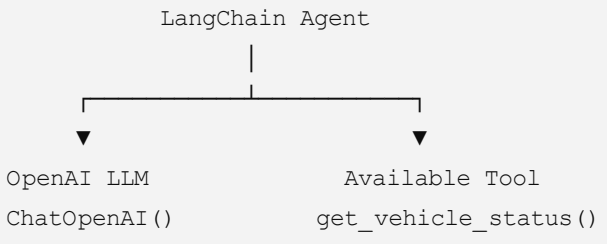
```python
from langchain.agents import create_agent
```

Then create the agent:

Python Run

```python
agent = create_agent(
    model=llm,

    tools=[
        get_vehicle_status
    ],

    system_prompt="""
    You are a BMW vehicle diagnostic assistant.

    When the user asks about a vehicle,
    use get_vehicle_status.

    Analyze the vehicle information.

    Classify the condition as:
    NORMAL, WARNING, or CRITICAL.

    Give a short recommendation.
    """
)
```

Now you have:



This line:

Python Run

```python
create_agent(...)
```

is where:

```
LLM + Tools + Instructions
```

are combined into an agent.

Step 11 — Create `run_agent()`

Add:

Python Run

```python
def run_agent(question: str):

    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": question
                }
            ]
        }
    )

    return result["messages"][-1].content
```

The most important line in the whole exercise is:

Python Run

```python
agent.invoke(...)
```

This starts the agent.

Conceptually:

```
agent.invoke()
      ↓
LLM
      ↓
Need a tool?
      ↓
Yes
      ↓
Tool call
      ↓
Tool result
      ↓
LLM again
```

Step 12 — Complete `agent.py`

Your complete file is now:

Python Run

```python
from dotenv import load_dotenv

from langchain.agents import create_agent
from langchain.tools import tool
from langchain_openai import ChatOpenAI


# ----------------------------------------
# Environment
# ----------------------------------------

load_dotenv()


# ----------------------------------------
# Tool
# ----------------------------------------

@tool
def get_vehicle_status(vehicle_id: str) -> str:
    """Get current BMW vehicle status."""

    print(
        f"Tool called for vehicle: {vehicle_id}"
    )

    if vehicle_id.upper() == "BMW1001":

        return """
        Vehicle: BMW1001
        Model: BMW iX
        Battery: 12%
        Temperature: 92 C
        Fault Code: P0A80
        """

    return "Vehicle not found"


# ----------------------------------------
# LLM
# ----------------------------------------
```

```python
llm = ChatOpenAI(
    model="gpt-5.6"
)


# ----------------------------------------
# Agent
# ----------------------------------------

agent = create_agent(
    model=llm,

    tools=[
        get_vehicle_status
    ],

    system_prompt="""
    You are a BMW vehicle diagnostic assistant.

    When the user asks about a vehicle,
    use get_vehicle_status.

    Analyze:
    - battery
    - temperature
    - fault code

    Classify the vehicle as:
    NORMAL, WARNING, or CRITICAL.

    Give a short recommendation.
    """
)


# ----------------------------------------
# Run Agent
# ----------------------------------------

def run_agent(question: str):

    result = agent.invoke(
        {
            "messages": [
                {
                    "role": "user",
                    "content": question
                }
            ]
        }
    )
```

```
    return result["messages"][-1].content
```

Notice that we added:

```python
print(
    f"Tool called for vehicle: {vehicle_id}"
)
```

This is useful in the lab because students can actually see that the LLM decided to call the tool.

Step 13 — Test the agent without FastAPI

Before adding FastAPI, test the agent.

Temporarily add to the bottom of `agent.py`:

```python
if __name__ == "__main__":

    answer = run_agent(
        "Check BMW1001"
    )

    print(answer)
```

Run:

```bash
python agent.py
```

You should see something similar to:

```
Tool called for vehicle: BMW1001
```

followed by:

```
CRITICAL

BMW1001 has a battery level of 12%,
temperature of 92 C and fault code P0A80.
```

```
Recommendation:
Stop driving safely and contact BMW service.
```

This proves the agent is working.

Step 14 — Understand what just happened

You wrote:

```python
run_agent(
    "Check BMW1001"
)
```

But you did **not write**:

```python
get_vehicle_status("BMW1001")
```

The LLM decided to do that.

The sequence was:

```
User:
"Check BMW1001"

        ↓

LangChain Agent

        ↓

LLM reads question

        ↓

LLM:
"I need vehicle information"

        ↓

Tool Call:

get_vehicle_status(
    vehicle_id="BMW1001"
)
```

```
        ↓

Tool returns:

Battery = 12%
Temperature = 92 C
Fault = P0A80

        ↓

LLM receives tool result

        ↓

LLM analyzes data

        ↓

CRITICAL

        ↓

Recommendation
```

This is the key learning outcome of the lab.

Step 15 — Add FastAPI

Create:

```
api.py
```

Add:

Python Run

```python
from fastapi import FastAPI
from pydantic import BaseModel

from agent import run_agent


app = FastAPI()


class AgentRequest(BaseModel):
    question: str
```

```python
@app.get("/")
def home():

    return {
        "message":
        "BMW LangChain Agent API"
    }


@app.post("/agent")
def ask_agent(
    request: AgentRequest
):

    answer = run_agent(
        request.question
    )

    return {
        "answer": answer
    }
```

FastAPI has only one responsibility:

```
Receive question
      ↓
run_agent()
      ↓
Return answer
```

Step 16 — Start FastAPI

Run:

```
Bash

uvicorn api:app --reload
```

Expected:

```
Uvicorn running on http://127.0.0.1:8000
```

Open Swagger:

```
http://127.0.0.1:8000/docs
```

Choose:

```
POST /agent
```

Click:

```
Try it out
```

Enter:

```
JSON
{
  "question": "Check BMW1001"
}
```

Click:

```
Execute
```

Expected response:

```
JSON
{
  "answer": "CRITICAL..."
}
```

Step 17 — Observe FastAPI terminal

This part is important during training.

The terminal should show:

```
Tool called for vehicle: BMW1001
```

That demonstrates:

```
FastAPI
   ↓
LangChain
   ↓
LLM
```

```
    ↓
```

FastAPI itself did not select the tool.

Step 18 — Add Streamlit

Create:

```
app.py
```

Add:

Python Run

```python
import requests
import streamlit as st


API_URL = "http://127.0.0.1:8000/agent"


st.title(
    "BMW LangChain Agent"
)


question = st.text_input(
    "Enter your question",
    placeholder="Check BMW1001"
)


if st.button("Ask Agent"):

    response = requests.post(
        API_URL,

        json={
            "question": question
        }
    )

    result = response.json()

    st.subheader(
        "Agent Response"
    )

    st.write(
```

```
        result["answer"]
    )
```

Step 19 — Start Streamlit

Keep FastAPI running.

Open another terminal.

Run:

```
Bash

streamlit run app.py
```

Browser opens around:

```
http://localhost:8501
```

You should see:

```
BMW LangChain Agent

Enter your question
[ Check BMW1001 ]

[ Ask Agent ]
```

Step 20 — Run complete application

Enter:

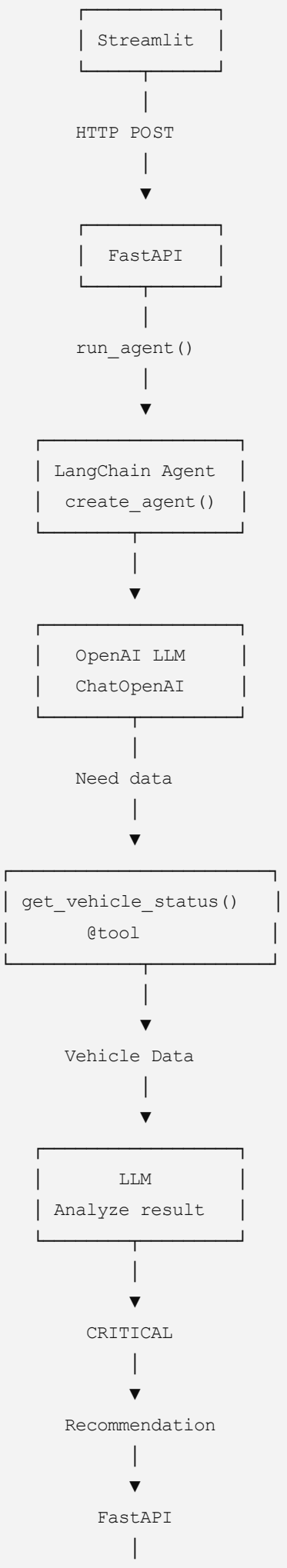
```
Check BMW1001
```

Click:

```
Ask Agent
```

Complete execution:

```
                USER
                  |
                  ▼
```


Streamlit
HTTP POST
FastAPI
run_agent()
LangChain Agent
create_agent()
OpenAI LLM
ChatOpenAI
Need data
get_vehicle_status()
@tool
Vehicle Data
LLM
Analyze result
CRITICAL
Recommendation
FastAPI

▼

Step 21 — Understand each file

File	Responsibility
`.env`	OpenAI API key
`pyproject.toml`	Dependencies
`agent.py`	LLM + Tool + Agent
`api.py`	FastAPI endpoint
`app.py`	Streamlit UI

The most important file is:

```
agent.py
```

because it contains:

```
Tool
 +
LLM
 +
Agent
```

Step 22 — Understand the four LangChain concepts

1. LLM

Python Run

```python
llm = ChatOpenAI(
    model="gpt-5.6"
)
```

Meaning:

```
Language Model
```

Its job:

```
Understand
Reason
Generate
Decide tool requirement
```

2. Tool

Python Run

```python
@tool
def get_vehicle_status(vehicle_id: str):
```

Its job:

```
Get external/business data
```

3. Agent

Python Run

```python
agent = create_agent(
    model=llm,
    tools=[get_vehicle_status]
)
```

Meaning:

```
Agent
=
LLM
+
Tools
+
Instructions
```

4. Invoke

Python Run

```python
agent.invoke(...)
```

Meaning:

```
Start agent execution
```

Step 23 — LLM versus Agent

This distinction should be clear before moving to the next lab.

Without an agent:

```
User
 ↓
LLM
 ↓
Answer
```

With an agent:

```
User
 ↓
Agent
 ↓
LLM
 ↓
Tool required?
 ↓
Tool
 ↓
LLM
 ↓
Answer
```

An LLM mainly:

```
understands + reasons + generates
```

An agent can:

```
understand
   +
decide
   +
use tools
   +
observe result
   +
continue reasoning
```

```
    +
produce final answer
```

Step 24 — Small exercise 1

Add another vehicle:

Python Run

```python
if vehicle_id.upper() == "BMW1002":

    return """
    Vehicle: BMW1002
    Model: BMW i4
    Battery: 82%
    Temperature: 38 C
    Fault Code: None
    """
```

Ask:

```
Check BMW1002
```

Expected classification:

```
NORMAL
```

Step 25 — Small exercise 2

Add:

Python Run

```python
if vehicle_id.upper() == "BMW1003":

    return """
    Vehicle: BMW1003
    Model: BMW X5
    Battery: 38%
    Temperature: 65 C
    Fault Code: None
    """
```

Ask:

```
What is the condition of BMW1003?
```

Observe what classification the model produces.

This demonstrates that:

```
Tool supplies facts
        ↓
LLM interprets facts
```

Step 26 — Small exercise 3

Ask something that does not require the vehicle tool:

```
What is an electric vehicle?
```

Ideally the agent can answer directly.

The flow is:

```
Question
   ↓
LLM
   ↓
Tool required?

NO
   ↓
Answer directly
```

Compare that with:

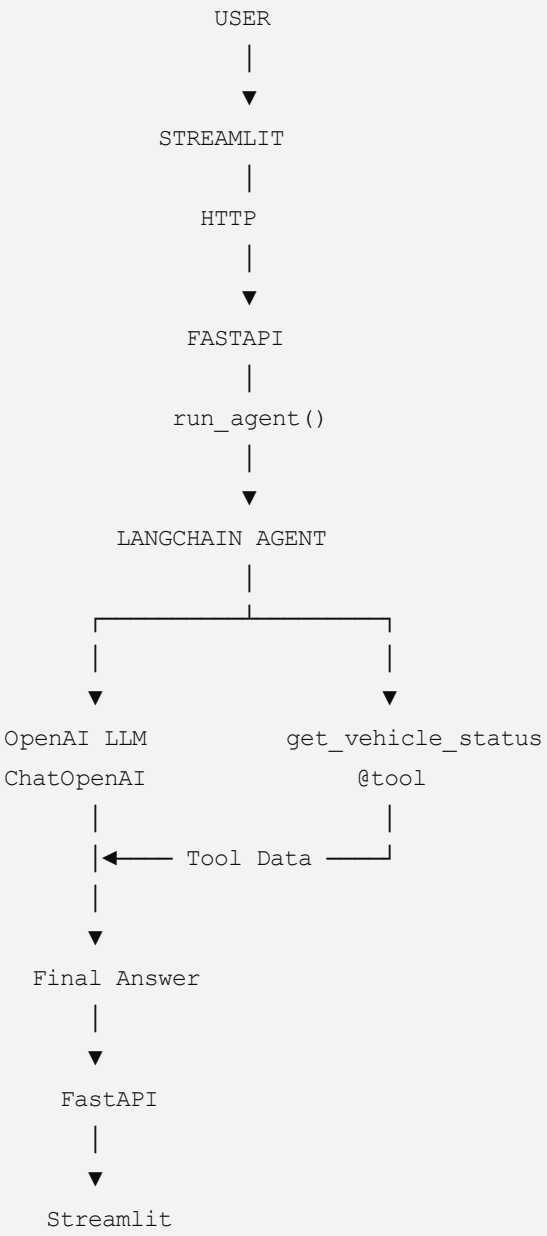
```
Check BMW1001
```

which should cause:

```
Tool required?

YES
   ↓
get_vehicle_status()
```

That comparison is one of the best ways to demonstrate what an **agent** is.

Final lab architecture



The entire first lab can therefore be summarized as:

Python Run

```python
# Tool
@tool
def get_vehicle_status(...):
    ...
```

Python Run

```python
# LLM
llm = ChatOpenAI(...)
```

Python Run

```python
# Agent
agent = create_agent(
    model=llm,
    tools=[get_vehicle_status]
)
```

Python Run

```python
# Execute
agent.invoke(...)
```

That is enough for the **first hands-on LangChain Agent exercise**. The natural second exercise would be to move vehicle data out of the hard-coded function into `vehicles.json`, and the third would introduce **multiple tools** so students can see the agent choose between them.